

6주차

집계와 그룹화

COUNT·SUM·AVG, GROUP BY, HAVING

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

이번 주를 마치면

목표 ①	집계 함수의 종류와 NULL 처리 방식을 정확히 이해한다
목표 ②	GROUP BY로 데이터를 묶어 경영 통계를 산출한다
목표 ③	WHERE와 HAVING의 차이를 설명하고 구분해 쓴다
목표 ④	ONLY_FULL_GROUP_BY 오류의 원인을 진단한다
목표 ⑤	취소 주문을 뺀 매출 집계처럼 조건과 집계를 결합한다

다운유통 5개 테이블로 경영 질문에 숫자로 답합니다

오늘 다루는 여섯 가지

1 집계 함수 (Aggregate Function)

2 GROUP BY — 그룹화

3 HAVING — 그룹 조건

4 실행 순서 다시 보기

5 취소 주문과 매출 집계 — 경영 관점

6 소계와 총계 — WITH ROLLUP

1. 집계 함수

여러 행의 값을 하나로 요약하는 함수

함수	기능	NULL 처리
COUNT(*)	전체 행의 수	NULL 포함하여 셈
COUNT(컬럼)	해당 컬럼이 NULL이 아닌 행의 수	NULL 제외
COUNT(DISTINCT 컬럼)	중복 제거한 값의 수	NULL 제외
SUM(컬럼)	합계	NULL 제외
AVG(컬럼)	평균	NULL 제외 (분모에서도 빠짐)
MIN(컬럼) / MAX(컬럼)	최솟값 / 최댓값	NULL 제외
GROUP_CONCAT(컬럼)	값을 하나의 문자열로 연결 (MySQL)	NULL 제외

그룹이 지정되지 않으면 테이블 전체를 하나의 그룹으로 본다

1. 집계 함수

COUNT(*)와 COUNT(컬럼)의 차이

```
1 SELECT COUNT(*)          AS 전체직원수,  -- 8
2       COUNT(salary)      AS 연봉입력수,  -- 7 (NULL 제외)
3       COUNT(DISTINCT dept) AS 부서수      -- 4
4 FROM   employees;

6 SELECT AVG(salary)        AS 평균1,
7       SUM(salary) / COUNT(*) AS 평균2,
8       AVG(IFNULL(salary, 0)) AS 평균3
9 FROM   employees;
```

1행

- 1 **COUNT(*) = 8**
행 자체를 세므로 NULL도 포함

2행

- 2 **COUNT(salary) = 7**
E008 조하늘의 NULL 제외

3행

- 3 **COUNT(DISTINCT) = 4**
같은 부서명은 한 번만

6-8행

- 4 같은 「평균」, 세 값
7명·8명 분모, NULL을 0으로

AVG는 NULL을 분자와 분모 모두에서 뺀다 — 0으로 볼지는 경영적 판단, 시험 단골 함정

1. 집계 함수

온라인 주문 세기와 상품 현황 요약

```
1  -- 주문 15건 중 emp_id가 NULL인 온라인 주문이 7건이므로
2  SELECT COUNT(*)      AS 전체주문,   -- 15 : 온라인 · 오프라인 모두
3         COUNT(emp_id) AS 오프라인주문 -- 8  (온라인 주문 제외)
4  FROM    orders;
5
6  -- 상품 표 전체를 한 줄로 요약 — 그룹이 없으니 결과는 1행
7  SELECT COUNT(*)      AS 상품수,      -- 10 : 등록된 상품 수
8         ROUND(AVG(price)) AS 평균가,   -- 32620 : 평균 판매가
9         MAX(price)     AS 최고가,     -- 189000 (공기청정기)
10        MIN(price)     AS 최저가,     -- 3200 (A5 스프링 노트)
11        SUM(stock)     AS 총재고      -- 모든 상품의 재고 수량 합계
12 FROM    products;
```

온라인 주문은 담당 직원이 없어 emp_id가 비어 있다 — COUNT(emp_id)는 오프라인 주문만 센다

2. GROUP BY

같은 값끼리 묶어 묶음마다 집계

1

SELECT category,

2

COUNT(*) AS 상품수,

3

ROUND(AVG(price)) AS 평균가

4

FROM products

5

GROUP BY category;

6

/* 결과

7

가전 2 115500

8

식품 3 19433

9

생활용품 3 9733

10

문구 2 3850 */

1

1행

1

그룹 기준 열

SELECT에 그대로 쓸 수 있다

2

2-3행

2

그룹마다 한 번씩

카테고리 안의 행 수·평균가

3

5행

3

category가 같은 행끼리

카테고리 수만큼 행이 나온다

4

7-10행

4

결과 4행

가전·식품·생활용품·문구

GROUP BY: 같은 값을 가진 행끼리 묶어, 묶음(그룹)마다 집계 함수를 한 번씩 계산한다

2. GROUP BY

한 열로 묶기, 두 열의 조합으로 묶기

등급별 고객 수 (VIP 2, GOLD 3, SILVER 5)

```
1 SELECT grade, COUNT(*) AS 고객수
2 FROM customers
3 GROUP BY grade;
```

다중 컬럼 그룹화: 등급 + 도시별 고객 수

```
1 SELECT grade, city, COUNT(*) AS 고객수
2 FROM customers
3 GROUP BY grade, city
4 ORDER BY grade, city;
```

두 열의 '조합'이 같은 행끼리 한 그룹(예: GOLD·서울) · 집계 결과는 순서가 보장되지 않으므로 ORDER BY로 정렬

2. GROUP BY

부서별 평균 연봉 — NULL이 집계에 미치는 영향

```
1 SELECT dept,  
2     COUNT(*)          AS 직원수,  
3     COUNT(salary)     AS 연봉입력수,  
4     ROUND(AVG(salary),1) AS 평균연봉  
5 FROM employees  
6 GROUP BY dept;  
7 /* 결과  
8   영업팀      3  3  403.3  
9   마케팅팀    2  2  410.0  
10  물류팀      2  2  390.0  
11  경영지원팀  1  0  NULL  */
```

2행

- 1 부서 인원
NULL과 상관없이 행 수

3행

- 2 연봉 입력 인원
salary가 있는 직원만

11행

- 3 경영지원팀
salary가 전부 NULL → AVG는 NULL

「직원 수」와 「연봉 데이터가 있는 직원 수」는 다른 질문 — 집계 함수가 그대로 보여 준다

SELECT에는 그룹 기준 열과 집계 함수만

```
1 SELECT grade, name, COUNT(*)
2 FROM customers
3 GROUP BY grade;
4 -- ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause
5 -- ... only_full_group_by 모드에서 발생
6
7 -- 해결 ① 그룹 기준에 추가
8 SELECT grade, city, COUNT(*) FROM customers GROUP BY grade, city;
9
10 -- 해결 ② 집계 함수로 감싸기
11 SELECT grade, GROUP_CONCAT(name) AS 고객목록, COUNT(*) AS 고객수
12 FROM customers GROUP BY grade;
```

1

1행

1

오류

name은 등급마다 여러 값

2

8행

2

해결 ①

그룹이 더 잘게 나뉜다

3

11-12행

3

해결 ②

GROUP_CONCAT도 집계

MySQL 8.0은 ONLY_FULL_GROUP_BY 모드가 기본으로 켜져 있어 규칙을 어기면 오류가 난다

3. HAVING

WHERE는 행, HAVING은 그룹

구분	WHERE	HAVING
적용 대상	개별 행(row)	그룹(group)
실행 시점	GROUP BY 이전	GROUP BY 이후
집계 함수 사용	불가	가능
용도	집계 대상 행을 선별	집계 결과를 선별

WHERE는 집계할 행을 고르고, HAVING은 집계가 끝난 결과를 고른다

3. HAVING

결제 금액 10만 원 이상인 주문

```
1 SELECT order_id,  
2    SUM(qty * unit_price) AS 결제금액  
3 FROM order_items  
4 GROUP BY order_id  
5 HAVING SUM(qty * unit_price) >= 100000;  
6 -- 결과: 1003(189000), 1008(114000), 1010(226000)  
7  
8 SELECT order_id, COUNT(*) AS 상품종수 FROM order_items  
9 GROUP BY order_id HAVING COUNT(*) >= 2;
```

1

2행

- 1 수량 × 단가를 합산
상세 한 줄의 금액을 주문별로

2

5행

- 2 묶은 뒤 거른다
합계 10만 원 이상인 그룹만

3

8-9행

- 3 두 종류 이상 담은 주문
묶음·교차 판매 분석에 활용

HAVING에 별칭은 MySQL에서는 되지만 다른 DBMS는 아닐 수 있다 — 집계 함수를 그대로 쓰는 것이 안전

3. HAVING

WHERE에는 집계 함수를 쓸 수 없다

✕ 오류 — WHERE 단계에는 아직 SUM이 없다

```
1 SELECT order_id, SUM(qty * unit_price) FROM order_items
2 WHERE SUM(qty * unit_price) >= 100000 GROUP BY order_id;
3 -- (X) ERROR 1111: Invalid use of group function
```



✓ 바른 예 — 집계 결과의 조건은 HAVING

```
1 SELECT order_id, SUM(qty * unit_price) AS 결제금액
2 FROM order_items
3 GROUP BY order_id
4 HAVING SUM(qty * unit_price) >= 100000;
```

WHERE는 행을 하나씩 보는 단계라 아직 SUM이 계산되지 않았다

4. 실행 순서 다시 보기

쓰는 순서와 실행 순서는 다르다

```
1  -- 절을 쓰는 순서와 실제 실행 순서는 다르다 — 번호(①~⑦)가 실행 순서
2  SELECT  grade, COUNT(*) AS cnt          -- ⑤ 결과 컬럼 생성, 별칭 부여
3  FROM    customers                      -- ① 테이블에서 행을 읽음
4  WHERE   city IS NOT NULL               -- ② 조건에 맞는 행만 남김
5  GROUP BY grade                        -- ③ grade 값이 같은 행끼리 묶음
6  HAVING  COUNT(*) >= 2                  -- ④ 2명 이상인 그룹만 남김
7  ORDER BY cnt DESC                     -- ⑥ 정렬 (별칭 사용 가능)
8  LIMIT   3;                           -- ⑦ 상위 3건
```

WHERE는 집계 전이라 집계 함수·별칭을 쓸 수 없고, HAVING은 집계 후라 집계 함수를 쓸 수 있다

4. 실행 순서 다시 보기

집계 질의의 처리 흐름 ① → ⑦



별칭은 ⑤ SELECT에서 생긴다 — 그보다 먼저 실행되는 WHERE에서는 쓸 수 없다

4. 실행 순서 다시 보기

실행 순서로 답하는 네 가지 질문

질문	답
WHERE에 별칭을 쓸 수 있는가?	아니오. WHERE(②)가 SELECT(⑤)보다 먼저 실행되기 때문
ORDER BY에 별칭을 쓸 수 있는가?	예. ORDER BY(⑥)가 SELECT(⑤) 이후이기 때문
WHERE에 집계 함수를 쓸 수 있는가?	아니오. 집계는 GROUP BY(③) 시점에 계산됨
HAVING만 쓰고 GROUP BY를 생략할 수 있는가?	예. 테이블 전체가 하나의 그룹이 됨

취소 주문을 빼지 않으면 매출이 부풀려진다

취소 주문 2건

01

다운유통의 주문 1005·1013은
「취소」 상태

들어오지 않은 돈

02

포함해 집계하면 실제 돈이
들어오지 않은 거래까지 실적에

집계 전에 선별

03

WHERE status <> '취소'
조건을 먼저 확인하는 습관

취소 제외는 그룹 조건이 아니라 행 조건 — HAVING이 아니라 WHERE에 둔다

월별 주문 건수 (취소 제외)

1

2

3

4

5

6

7

8

9

10

11

12

13

```
1 SELECT DATE_FORMAT(order_date, '%Y-%m') AS 주문월,  
2         COUNT(*) AS 주문건수  
3 FROM orders  
4 WHERE status <> '취소'  
5 GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
6 ORDER BY 주문월;  
7 /* 결과  
8 2026-01 2  
9 2026-02 2 ← 취소된 1005가 빠져 3건이 아니라 2건  
10 2026-03 3  
11 2026-04 2  
12 2026-05 2 ← 취소된 1013 제외  
13 2026-06 2 */
```

1

2

3

4

5

6

1행

1 연-월 문자열로
'2026-01' 형태

4행

2 집계 전에 행 선별
<> 는 '같지 않다'

5행

3 SELECT와 같은 식
같은 식으로 묶어야 한다

6행

4 별칭으로 정렬
ORDER BY는 SELECT 다음

5. 취소 주문과 매출 집계

상태별 주문 분포 — 취소가 몇 건인지 먼저

상태(status)	건수	WHERE status <> '취소' 적용 시
배송완료	9	남는다
배송중	3	남는다
취소	2	빠진다 (1005·1013)
주문접수	1	남는다

금액 기준 매출액은 orders와 order_items를 조인해야 구한다(7주차) — 이번 주는 「취소를 먼저 거르고 집계」만

```
SELECT status, COUNT(*) AS 건수 FROM orders GROUP BY status;
```

6. WITH ROLLUP

소계·총계 행을 자동으로 덧붙인다

```
1 SELECT IFNULL(grade, '전체합계') AS 등급,  
2        COUNT(*)                AS 고객수  
3 FROM   customers  
4 GROUP BY grade WITH ROLLUP;  
5 -- 결과: VIP 2 · GOLD 3 · SILVER 5 + 마지막에 전체합계 10 행  
6  
7 SELECT category, name, SUM(stock) AS 재고  
8 FROM   products  
9 GROUP BY category, name WITH ROLLUP;
```

1

1행

1

총계 행에 이름 붙이기

총계 행의 grade는 NULL로 나온다

2

4행

2

등급별 3행 + 총계 1행

3

9행

3

다중 그룹의 소계

카테고리 소계(name NULL) + 총계

이번 주 집계 문법 모음

하려는 일	SQL	기억할 점
전체 행 수	COUNT(*)	NULL도 센다
값이 있는 행 수	COUNT(salary)	NULL 제외
중복 없는 값의 수	COUNT(DISTINCT dept)	NULL 제외
NULL을 0으로 보고 평균	AVG(IFNULL(salary, 0))	경영적 판단
그룹에 조건	HAVING COUNT(*) >= 2	집계 후 필터
소계·총계 행	GROUP BY grade WITH ROLLUP	총계 행은 NULL
조건부 개수 세기	SUM(CASE WHEN status = '취소' THEN 1 ELSE 0 END)	
같은 결과	COUNT(CASE WHEN 조건 THEN 1 END)	ELSE 없으면 NULL

SUM(CASE ...)은 전체 주문 중 취소 건수를 한 질의 안에서 함께 세는 표준 패턴

집계 질의에서 자주 만나는 다섯 가지

■ 증상

! ERROR 1111: Invalid use of group function

! ERROR 1055 ... not in GROUP BY clause

! 평균이 생각과 다르다

! 매출 실적이 부풀려졌다

! WHERE에서 별칭을 못 찾는다

■ 원인

→ WHERE에 SUM 같은 집계 함수

→ 그룹 기준도 집계도 아닌 열을 SELECT

→ AVG는 NULL을 분모에서도 뺀다

→ 취소 주문까지 집계에 포함

→ WHERE(②)가 SELECT(⑤)보다 먼저

■ 해결

→ ✓ 집계 결과 조건은 HAVING으로

→ ✓ GROUP BY에 추가하거나 집계 함수로

→ ✓ 0으로 볼 거면 AVG(IFNULL(컬럼,0))

→ ✓ WHERE status <> '취소'를 먼저

→ ✓ 식을 그대로 쓰거나 ORDER BY에서

확인 퀴즈

집계 함수와 NULL 처리에 대한 설명으로 옳은 것은?

① COUNT(*)는 NULL을 제외하고 센다

② COUNT(컬럼)은 NULL을 포함하여 센다

③ AVG(컬럼)은 NULL을 제외하고 평균을 계산한다

④ SUM(컬럼)은 NULL을 0으로 간주한다

정답 및 해설

정답

③ AVG(컬럼)은 NULL을 제외하고 평균을 계산한다

해설

COUNT(*)만 NULL을 포함하며, COUNT(컬럼)·SUM·AVG 등 나머지 집계 함수는 NULL을 제외합니다.

AVG(salary)는 연봉 협상 중인 E008을 빼고 평균을 냅니다.

핵심 요약

핵심 ①	COUNT(*)는 NULL 포함, COUNT(컬럼)·다른 집계는 NULL 제외
핵심 ②	GROUP BY 시 SELECT에는 그룹 기준 열과 집계 함수만
핵심 ③	WHERE는 집계 전 행 필터, HAVING은 집계 후 그룹 필터
핵심 ④	매출·실적 집계는 취소 주문을 WHERE로 먼저 제외
핵심 ⑤	WITH ROLLUP은 소계·총계 행을 자동으로 추가

E008의 salary NULL이 AVG에서 빠진다 — 시험 단골 함정

다음 주차 예고

7주차 · 조인 (JOIN)

01

조인의 개념

나뉜 테이블을
공통 컬럼으로 연결

02

INNER JOIN

양쪽 모두 맞는 행
3개 이상 테이블 조인

03

OUTER JOIN

주문 없는 고객까지
ON과 WHERE의 차이

04

SELF·CROSS

직원-상급자
카티전 곱

6주차 마무리

학습 마무리

이번 주 실습 문제를 다운유통 데이터로 직접 풀어 보세요

✓ 집계 함수와 NULL 처리

✓ GROUP BY와 ONLY_FULL_GROUP_BY

✓ WHERE vs HAVING과 실행 순서

✓ 취소 제외 집계와 WITH ROLLUP

이어나는 활동

6주차 실습 10문제 · 퀴즈 5